STAGE 1

In this stage I build a tokenizer from scratch for data preparation and sampling purposes.

Here for dataset I am using WikiText-2 from huggingface and TinyStories.

Steps followed usually :

1. Split the text into individual word and subwords
2. Convert these tokens into token IDs.

```
! pip install datasets
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from data
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from d
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datase
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datase
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from dat
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from d
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from hugging
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from htt
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from pyth
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from ma
```

```
from datasets import load_dataset

wiki = load_dataset("wikitext", "wikitext-2-raw-v1")
print(wiki)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggi
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to ena
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the
```

README.md: 10.5k/? [00:00<00:00, 124kB/s]

wikitext-2-raw-v1/test-00000-of-00001.pa(…): 100% 733k/733k [00:01<00:00, 1.95MB/s]

wikitext-2-raw-v1/train-00000-of-00001.p(…): 100% 6.36M/6.36M [00:00<00:00, 31.9MB/s]

wikitext-2-raw-v1/validation-00000-of-00(…): 100% 657k/657k [00:00<00:00, 2.24MB/s]

Generating test split: 100% 4358/4358 [00:00<00:00, 9556.03 examples/s]

Generating train split: 100% 36718/36718 [00:00<00:00, 110248.96 examples/s]

Generating validation split: 100% 3760/3760 [00:00<00:00, 77160.78 examples/s]

```
DatasetDict({
    test: Dataset({
        features: ['text'],
        num_rows: 4358
    })
    train: Dataset({
        features: ['text'],
        num_rows: 36718
    })
    validation: Dataset({
        features: ['text'],
        num_rows: 3760
    })
})
```

We can see here its a dataset dict. For encoder we only need the train part.

```
wiki_text = wiki['train']['text']
print(repr(wiki_text[0]))
```

```
''
```

```
for i in range(5):
  print(repr(wiki_text[i]))
```

```
''
' = Valkyria Chronicles III = \n'
''
```

```
' Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァルキュリア3 , lit . Valkyr
" The game began development in 2010 , carrying over a large portion of the work done on Valk
```

Alright first i thought something was wrong but it seems everything is working correctly.

```
stories = load_dataset("roneneldan/TinyStories")
stories_text = stories['train']['text']
for i in range(5):
  print(repr(stories_text[i]))
```

```
README.md:        1.06k/?  [00:00<00:00,  44.1kB/s]

data/train-00000-of-00004-                                    249M/249M  [00:02<00:00,  168MB/s]

2d5a1467fff108(…): 100%

data/train-00001-of-00004-                                    248M/248M  [00:02<00:00,  214MB/s]

5852b56a2bd28f(…): 100%

data/train-00002-of-00004-                                    246M/246M  [00:02<00:00,  241MB/s]

a26307300439e9(…): 100%

data/train-00003-of-00004-                                    248M/248M  [00:02<00:00,  1.24GB/s]

d243063613e5a0(…): 100%

data/validation-00000-of-00001-                               9.99M/9.99M  [00:00<00:00,  50.2MB/s]

869c898b5(…): 100%

Generating train split: 100%                  2119719/2119719  [00:27<00:00,  109426.38 examples/s]

Generating validation split: 100%                21990/21990  [00:00<00:00,  70712.01 examples/s]
'One day, a little girl named Lily found a needle in her room. She knew it was difficult to p
'Once upon a time, there was a little car named Beep. Beep loved to go fast and play in the s
'One day, a little fish named Fin was swimming near the shore. He saw a big crab and wanted t
'Once upon a time, in a land full of trees, there was a little cherry tree. The cherry tree w
'Once upon a time, there was a little girl named Lily. Lily liked to pretend she was a popula
```

```
print(type(stories_text))
```

```
<class 'datasets.arrow_dataset.Column'>
```

Looks good to me now i think i can move forward.

```
for i in range(10):
  print(repr(wiki_text[i]))
```

```
''
' = Valkyria Chronicles III = \n'
''
' Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァルキュリア3 , lit . Valkyr
" The game began development in 2010 , carrying over a large portion of the work done on Valk
" It met with positive sales in Japan , and was praised by both Japanese and western critics
''
' = = Gameplay = = \n'
```

```
''
" As with previous Valkyira Chronicles games , Valkyria Chronicles III is a tactical role @-@
```

It seems as if theres a lot of empty spaces i should clean them first.

```
wiki_clean = [item for item in wiki_text if item.strip() != ' ']
for i in range(10):
  print(wiki_clean[i])
```

```
 = Valkyria Chronicles III =


 Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァルキュリア3 , lit . Valkyri

 The game began development in 2010 , carrying over a large portion of the work done on Valky

 It met with positive sales in Japan , and was praised by both Japanese and western critics .


 = = Gameplay = =


 As with previous Valkyira Chronicles games , Valkyria Chronicles III is a tactical role @-@
```

Ideas

I am keeping the *= = = Titles = = =* sections in hopes my model will learn these mean headings or titles and might print

= = = Knight's Voyage = = =

or something like that

```
"dataset = wiki_clean + stories_text # I forgot to convert both types hehe :p "
```

```
'dataset = wiki_clean + stories_text # I forgot to convert both types hehe :p '
```

```
stories_text = stories_text = stories['train']['text'][:]
print(type(stories_text))
for i in range(5):
  print(stories_text[i])
```

```
<class 'list'>
One day, a little girl named Lily found a needle in her room. She knew it was difficult to pl

Lily went to her mom and said, "Mom, I found this needle. Can you share it with me and sew my

Together, they shared the needle and sewed the button on Lily's shirt. It was not difficult f
Once upon a time, there was a little car named Beep. Beep loved to go fast and play in the su

One day, Beep was driving in the park when he saw a big tree. The tree had many leaves that w

Beep played with the falling leaves all day. When it was time to go home, Beep knew he needed
One day, a little fish named Fin was swimming near the shore. He saw a big crab and wanted to
```

```
Fin felt sad but wanted to help the crab feel better. He swam away and thought of a plan. He

The sun heard Fin's call and shone its warm light on the shore. The crab started to feel bett
Once upon a time, in a land full of trees, there was a little cherry tree. The cherry tree wa

One day, the cherry tree felt a tickle in its branches. It was a little spring wind. The wind

As time went on, the cherry tree grew more and more cherries. All the animals in the land cam
Once upon a time, there was a little girl named Lily. Lily liked to pretend she was a popular

One day, while playing in the castle, Lily found a big cobweb. The cobweb was in the way of h

Lily asked her friends, the cat and the dog, to help her. They all worked together to clean t
```

```
dataset = wiki_clean + stories_text
```

```
print(len(dataset))
```

```
2156437
```

Sweet

Now we build the tokenizer.

But before that I will make a .py file.

PS: Now I think i should remove " *==== Titles ===* "

I can include them in finetuning

```
dataset = [item for item in dataset if not item.strip().startswith("=")]
```

```
#A pretty stupid line
print("Searching if dataset has lines starting with '= = = Title = = =' ")
for i in range(len(dataset)):
  if dataset[i].strip().startswith("="):
    print(dataset[i])

print(". . .\n")

print("Searching complete lets look at some samples")
for i in range(5):
  print(dataset[i])
```

```
Searching if dataset has lines starting with '= = = Title = = ='
. . .

Searching complete lets look at some samples


 Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァルキュリア3 , lit . Valkyri

 The game began development in 2010 , carrying over a large portion of the work done on Valky

 It met with positive sales in Japan , and was praised by both Japanese and western critics .
```

LOL it didnt print anything but contents did print stuff

For the time being i save my dataset

```
"""final_dataset = " ".join(dataset)
print(final_dataset[1:9])"""
```

```
'final_dataset = " ".join(dataset)\nprint(final_dataset[1:9])'
```

```
"""print(final_dataset[:100])"""
```

```
'print(final_dataset[:100])'
```

tiktoken is a library that helps us use BPE. We instantiate BPE.

```
# ! pip install tiktoken
```

```
"""import importlib
import tiktoken"""
```

```
'import importlib\nimport tiktoken'
```

```
"""tokenizer = tiktoken.get_encoding("gpt2")"""
```

```
'tokenizer = tiktoken.get_encoding("gpt2")'
```

Every tokenizer has encode method and decode method.

This is good enough but I'd like to make my own tokenizer.

```
print(len(dataset))
```

```
2150225
```

```
#STEP 1
vocab = []

for line in range(394999):
    words = dataset[line].split()
    for word in words:
        vocab.append(" ".join(list(word)) + " </w>")
```

```
print(len(vocab))
```

```
66360319
```

```
print(vocab[10])
```

```
戦 場 の ヴ ァ ル キ ュ リ ア 3 </w>
```

What the-

```
print(vocab[1010])
```

```
c h a r a c t e r s </w>
```

```
example = len(set(vocab))
print(example)
```

```
191468
```

```
from collections import Counter
```

```
print(type(vocab))
```

```
<class 'list'>
```

```
word_freq = Counter(vocab)
```

```
print(type(word_freq))
```

```
<class 'collections.Counter'>
```

Cant work with this need to make changes

```
vocab_freq = Counter(vocab)
print(len(vocab_freq))
print(type(vocab_freq))
```

```
191468
<class 'collections.Counter'>
```

```
for i, (word, freq) in enumerate(vocab_freq.items()):
    print(word, ":", freq)
    if i == 10:
        break
```

```
S e n j ō </w> : 5
n o </w> : 38479
V a l k y r i a </w> : 53
3 </w> : 8408
: </w> : 3742
U n r e c o r d e d </w> : 1
C h r o n i c l e s </w> : 46
( </w> : 11744
J a p a n e s e </w> : 256
戦 場 の ヴ ァ ル キ ュ リ ア 3 </w> : 3
, </w> : 99786
```

```
"""import re, collections
def get_stats(vocab):
```

```
  pairs = collections.defaultdict(int)
  for word, freq in vocab.items():
    symbols = word.split()
  for i in range(len(symbols)-1):
    pairs[symbols[i],symbols[i+1]] += freq
  return pairs

def merge_vocab(pair, v_in):
  v_out = {}
  bigram = re.escape(' '.join(pair))
  p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
  for word in v_in:
    w_out = p.sub(''.join(pair), word)
    v_out[w_out] = v_in[word]
  return v_out
vocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,
'n e w e s t </w>':6, 'w i d e s t </w>':3}


num_merges = 10
for i in range(num_merges):
  pairs = get_stats(vocab)
  print(pairs)
  best = max(pairs, key=pairs.get())
  vocab = merge_vocab(best, vocab)
  print(best)

# print(vocab)"""
```

```
<>:13: SyntaxWarning: invalid escape sequence '\S'
<>:13: SyntaxWarning: invalid escape sequence '\S'
/tmp/ipykernel_14003/4246427281.py:13: SyntaxWarning: invalid escape sequence '\S'
  p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
'import re, collections\ndef get_stats(vocab):\n  pairs = collections.defaultdict(int)\n  for
word, freq in vocab.items():\n    symbols = word.split()\n  for i in range(len(symbols)-1):\n
pairs[symbols[i],symbols[i+1]] += freq\n  return pairs\n\ndef merge_vocab(pair, v_in):\n  v_o
ut = {}\n  bigram = re.escape(' '.join(pair))\n  p = re.compile(r'(?<!\\S)' + bigram + r'(?!
\\S)')\n  for word in v_in:\n    w_out = p.sub(''.join(pair), word)\n    v_out[w_out] = v_in
[word]\n  return v_out\nvocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,\n'n e w e s t </w>':
6, 'w i d e s t </w>':3}\n\n\nnum_merges = 10\nfor i in range(num_merges):\n  pairs = get_sta
ts(vocab)\n  print(pairs)\n  best = max(pairs, key=pairs.get())\n  vocab = merge_vocab(best,
```

```
for word in range(5):
  subword = vocab[word].split()
  print(vocab[word])
  print(subword)
```

```
S e n j ō </w>
['S', 'e', 'n', 'j', 'ō', '</w>']
n o </w>
['n', 'o', '</w>']
V a l k y r i a </w>
['V', 'a', 'l', 'k', 'y', 'r', 'i', 'a', '</w>']
3 </w>
['3', '</w>']
: </w>
[':', '</w>']
```

```
def get_pair(vocab_freq):
```

```
  pairs = {}

  for word,freq in vocab_freq.items():
    subword = word.split()
    for i in range(len(subword)-1):
      pair = (subword[i],subword[i+1])
      pairs[pair] = pairs.get(pair, 0) + freq

  return pairs
```

```
pairs = get_pair(vocab_freq)
```

```
for i, (subword, freq) in enumerate(pairs.items()):
    print(subword, ":", freq)
    if i == 10:
        break
```

```
('S', 'e') : 8242
('e', 'n') : 2054283
('n', 'j') : 30517
('j', 'ō') : 6
('ō', '</w>') : 34
('n', 'o') : 732893
('o', '</w>') : 3358850
('V', 'a') : 1213
('a', 'l') : 1161382
('l', 'k') : 137616
('k', 'y') : 76296
```

```
print(len(pairs))
```

```
6650
```

```
best = max(pairs, key=pairs.get)
print(best)
```

```
('e', '</w>')
```

```
for i, (word,freq) in enumerate(vocab_freq.items()):
  print(word,type(word), ":", freq)
  if i == 10:
    break
```

```
S e n j ō </w> <class 'str'> : 5
n o </w> <class 'str'> : 38479
V a l k y r i a </w> <class 'str'> : 53
3 </w> <class 'str'> : 8408
: </w> <class 'str'> : 3742
U n r e c o r d e d </w> <class 'str'> : 1
C h r o n i c l e s </w> <class 'str'> : 46
( </w> <class 'str'> : 11744
J a p a n e s e </w> <class 'str'> : 256
戦 場 の ヴ ァ ル キ ュ リ ア 3 </w> <class 'str'> : 3
, </w> <class 'str'> : 99786
```

```
"""Original merge operation
def merge_vocab(word_pair, vocab_in):
```

```
  keysub = " ".join(word_pair)
  replacement = "".join(word_pair)"""

#i should not modify old vocab i need to make new list of strs
""" for word,freq in vocab_in :
    temp_char_stream = word.split()"""
    #FORGOT .items again

    # REPLACE for loop here, the list size updates so i need to replace it
    # using pointer i in while loop instead so that it ensures i is less than
    # length of the you know.
"""
    for item in range(len(temp_char_stream)-1):
      comparison = (temp_char_stream[item]+" "+temp_char_stream[item+1])"""

      # If match i will append new token (the merged one rather than the updating)
"""
      if comparison == keysub:
        temp_char_stream[item] = replacement
        temp_char_stream.pop(item+1)
      final_word = " ".join(temp_char_stream)

  vocab_in[final_word] += 1"""
```

```
'\n      if comparison == keysub:\n        temp_char_stream[item] = replacement\n        temp
_char_stream.pop(item+1)\n      final_word = " ".join(temp_char_stream)\n  \n  vocab_in[final
_word] += 1'
```

```
def merge_vocab(word_pair, vocab_in):
  keysub = " ".join(word_pair)
  replacement = "".join(word_pair)

  new_vocab = {}

  for word,freq in vocab_in.items():
        item = 0
        new_token = []
        temp_char_stream = word.split()

        while item < len(temp_char_stream)-1:
            comparison = (temp_char_stream[item])+" "+temp_char_stream[item+1]

            if comparison == keysub:
              new_token.append(replacement)
              item += 2

            else:
              new_token.append(temp_char_stream[item])
              item += 1

        final_word = " ".join(new_token)
        new_vocab[final_word] = freq

  return new_vocab
```

```
new_freqs = merge_vocab(best, vocab_freq)
```

```
for i, (key,freq) in enumerate(new_freqs.items()):
  if "".join(best) in key:
    print(key, ":", freq)
    if i == 10:
      break
```

```
" I n e 96/w> : 1
```

```
    Start coding or generate with AI.
```